

COMPILER DESIGN CODES

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isKeyword(char *word) {
    char *keywords[] = {
        "int", "float", "char", "if", "else",
        "while", "for", "return", "void"
    };

    int n = sizeof(keywords) / sizeof(keywords[0]);

    for (int i = 0; i < n; i++) {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main() {
    char input[500];
    int i = 0;

    printf("Enter source code:\n");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0') {

        if (isspace(input[i])) {
            i++;
            continue;
        }

        /* Identifier or keyword */
        if (isalpha(input[i]) || input[i] == '_') {
            char word[50];
            int j = 0;

            while (isalnum(input[i]) || input[i] == '_') {
                word[j++] = input[i++];
            }
        }
    }
}
```

```

    }

    word[j] = '\0';

    if (isKeyword(word))
        printf("%s -> KEYWORD\n", word);
    else
        printf("%s -> IDENTIFIER\n", word);
}

/* Number */
else if (isdigit(input[i])) {
    char number[50];
    int j = 0;

    while (isdigit(input[i])) {
        number[j++] = input[i++];
    }

    number[j] = '\0';

    printf("%s -> NUMBER\n", number);
}

/* Operators */
else if (input[i] == '+' || input[i] == '-' ||
        input[i] == '*' || input[i] == '/' ||
        input[i] == '=') {

    printf("%c -> OPERATOR\n", input[i]);
    i++;
}

/* Separators */
else if (input[i] == ';' || input[i] == ',' ||
        input[i] == '(' || input[i] == ')' ||
        input[i] == '{' || input[i] == '}') {

    printf("%c -> SEPARATOR\n", input[i]);
    i++;
}

else {
    printf("%c -> UNKNOWN\n", input[i]);
}

```

```

        i++;
    }
}

return 0;
}
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 20

char productions[MAX][MAX];
char first[MAX][MAX];
char follow[MAX][MAX];

int n;

void add(char set[], char c) {
    int i;

    for (i = 0; set[i] != '\0'; i++) {
        if (set[i] == c)
            return;
    }

    int len = strlen(set);
    set[len] = c;
    set[len + 1] = '\0';
}

int isNonTerminal(char c) {
    return c >= 'A' && c <= 'Z';
}

void FIRST(char result[], char symbol) {

    int i, j;

    if (!isNonTerminal(symbol)) {
        add(result, symbol);
        return;
    }

```

```

for (i = 0; i < n; i++) {

    if (productions[i][0] == symbol) {

        char *rhs = strchr(productions[i], '>');

        if (rhs == NULL)
            continue;

        rhs++;

        for (j = 0; rhs[j] != '\0'; j++) {

            if (rhs[j] == '#') {
                add(result, '#');
                break;
            }

            if (!isNonTerminal(rhs[j])) {
                add(result, rhs[j]);
                break;
            }

            char temp[MAX] = "";
            FIRST(temp, rhs[j]);

            int hasEpsilon = 0;

            for (int k = 0; temp[k] != '\0'; k++) {
                if (temp[k] == '#')
                    hasEpsilon = 1;
                else
                    add(result, temp[k]);
            }

            if (!hasEpsilon)
                break;

            if (rhs[j + 1] == '\0')
                add(result, '#');
        }
    }
}

```

```

int main() {

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (use # for epsilon):\n");

    for (int i = 0; i < n; i++)
        scanf("%s", productions[i]);

    printf("\nFIRST sets:\n");

    for (int i = 0; i < n; i++) {

        char nt = productions[i][0];
        char result[MAX] = "";

        FIRST(result, nt);

        printf("FIRST(%c) = { ", nt);

        for (int j = 0; result[j] != '\0'; j++)
            printf("%c ", result[j]);

        printf("\n");
    }

    return 0;
}

#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c) {
    stack[++top] = c;
}

char pop() {
    return stack[top--];
}

```

```

void printStack() {
    for (int i = 0; i <= top; i++)
        printf("%c", stack[i]);

    printf("\n");
}

int main() {

    char input[100];

    printf("Enter expression using i for identifier: ");
    scanf("%s", input);

    strcat(input, "$");

    push('$');
    push('E');

    int ip = 0;

    printf("\nStack\tInput\tAction\n");

    while (top >= 0) {

        printStack();

        printf("%s\t", input + ip);

        char X = stack[top];
        char a = input[ip];

        if (X == a) {

            pop();
            ip++;

            printf("Match %c\n", a);
        }

        else if (X == 'E') {

            pop();

```

```

    push('R');
    push('T');

    printf("E -> TR\n");
}

else if (X == 'R') {

    if (a == '+') {
        pop();
        push('R');
        push('T');
        push('+');

        printf("R -> +TR\n");
    }

    else if (a == ')' || a == '$') {
        pop();

        printf("R -> epsilon\n");
    }

    else {
        printf("ERROR\n");
        return 0;
    }
}

else if (X == 'T') {

    pop();
    push('Y');
    push('F');

    printf("T -> FY\n");
}

else if (X == 'Y') {

    if (a == '*') {
        pop();
        push('Y');
        push('F');
    }
}

```

```

        push('*');

        printf("Y -> *FY\n");
    }

    else if (a == '+' || a == '-' || a == '$') {
        pop();

        printf("Y -> epsilon\n");
    }

    else {
        printf("ERROR\n");
        return 0;
    }
}

else if (X == 'F') {

    if (a == 'i') {
        pop();
        push('i');

        printf("F -> i\n");
    }

    else if (a == '(') {
        pop();
        push('(');
        push('E');
        push('(');

        printf("F -> (E)\n");
    }

    else {
        printf("ERROR\n");
        return 0;
    }
}

else {
    printf("ERROR\n");
    return 0;
}

```

```

    }

    if (stack[top] == '$' && input[ip] == '$')
        break;
}

printf("\nString accepted by predictive parser.\n");

return 0;
}

#include <stdio.h>
#include <string.h>
#include <ctype.h>

char expression[100];
int tempCount = 1;

void generateCode(char op, char a, char b) {

    printf("t%d = %c %c %c\n",
        tempCount, a, op, b);

    tempCount++;
}

int main() {

    printf("Enter expression: ");
    scanf("%s", expression);

    printf("\nThree Address Code:\n");

    char temp[100];

    strcpy(temp, expression);

    /* Multiplication and division */
    for (int i = 0; temp[i] != '\0'; i++) {

        if (temp[i] == '*' || temp[i] == '/') {

            char left = temp[i - 1];
            char right = temp[i + 1];

```

```

generateCode(temp[i], left, right);

char newTemp = '0' + tempCount - 1;

temp[i - 1] = newTemp;

int j = i;

while (temp[j + 2] != '\0') {
    temp[j] = temp[j + 2];
    j++;
}

temp[j] = '\0';

i--;
}
}

```

```

/* Addition and subtraction */
for (int i = 0; temp[i] != '\0'; i++) {

    if (temp[i] == '+' || temp[i] == '-') {

        char left = temp[i - 1];
        char right = temp[i + 1];

        generateCode(temp[i], left, right);

        char newTemp = '0' + tempCount - 1;

        temp[i - 1] = newTemp;

        int j = i;

        while (temp[j + 2] != '\0') {
            temp[j] = temp[j + 2];
            j++;
        }

        temp[j] = '\0';

        i--;
    }
}

```

```

    }
}

return 0;
}

#include <stdio.h>
#include <string.h>

int main() {

    char op[10];
    char left[20], right[20];

    printf("Enter assignment in the form: a = b + 0\n");
    scanf("%s = %s %s %s", left, left, right, op, right);

    return 0;
}

#include <stdio.h>
#include <string.h>

int main() {

    char lhs[20], op[5], a[20], b[20];

    printf("Enter expression (example: x = y + 0): ");
    scanf("%s = %s %s %s", lhs, a, op, b);

    printf("\nOriginal Code:\n");
    printf("%s = %s %s %s\n", lhs, a, op, b);

    printf("\nOptimized Code:\n");

    if (strcmp(op, "+") == 0 && strcmp(b, "0") == 0) {
        printf("%s = %s\n", lhs, a);
    }

    else if (strcmp(op, "**") == 0 && strcmp(b, "1") == 0) {
        printf("%s = %s\n", lhs, a);
    }

    else if (strcmp(op, "**") == 0 && strcmp(b, "0") == 0) {

```

```

        printf("%s = 0\n", lhs);
    }

    else {
        printf("%s = %s %s %s\n",
            lhs, a, op, b);
    }

    return 0;
}

#include <stdio.h>
#include <string.h>

#define MAX 100

struct Symbol {
    char name[30];
    char type[20];
    char dataType[20];
    int address;
};

struct Symbol table[MAX];
int count = 0;

/* Search for a symbol */
int searchSymbol(char name[]) {
    for (int i = 0; i < count; i++) {
        if (strcmp(table[i].name, name) == 0)
            return i;
    }

    return -1;
}

/* Insert a new symbol */
void insertSymbol() {

    char name[30];
    char type[20];
    char dataType[20];
    int address;

```

```

printf("\nEnter symbol name: ");
scanf("%s", name);

/* Check duplicate */
if (searchSymbol(name) != -1) {
    printf("Error: Symbol already exists.\n");
    return;
}

printf("Enter symbol type: ");
scanf("%s", type);

printf("Enter data type: ");
scanf("%s", dataType);

printf("Enter memory address: ");
scanf("%d", &address);

strcpy(table[count].name, name);
strcpy(table[count].type, type);
strcpy(table[count].dataType, dataType);
table[count].address = address;

count++;

printf("Symbol inserted successfully.\n");
}

/* Display symbol table */
void displayTable() {

    if (count == 0) {
        printf("\nSymbol table is empty.\n");
        return;
    }

    printf("\n-----\n");
    printf("%-15s %-15s %-15s %-10s\n",
        "Name", "Type", "Data Type", "Address");
    printf("-----\n");

    for (int i = 0; i < count; i++) {

        printf("%-15s %-15s %-15s %-10d\n",

```

```

        table[i].name,
        table[i].type,
        table[i].dataType,
        table[i].address);
    }

    printf("-----\n");
}

/* Search for a symbol */
void search() {

    char name[30];

    printf("\nEnter symbol to search: ");
    scanf("%s", name);

    int index = searchSymbol(name);

    if (index == -1) {
        printf("Symbol not found.\n");
    }
    else {
        printf("\nSymbol Found\n");
        printf("Name      : %s\n", table[index].name);
        printf("Type       : %s\n", table[index].type);
        printf("Data Type  : %s\n", table[index].dataType);
        printf("Address    : %d\n", table[index].address);
    }
}

/* Delete a symbol */
void deleteSymbol() {

    char name[30];

    printf("\nEnter symbol to delete: ");
    scanf("%s", name);

    int index = searchSymbol(name);

    if (index == -1) {
        printf("Symbol not found.\n");
        return;
    }

```

```

    }

    for (int i = index; i < count - 1; i++) {
        table[i] = table[i + 1];
    }

    count--;

    printf("Symbol deleted successfully.\n");
}

int main() {

    int choice;

    while (1) {

        printf("\n===== SYMBOL TABLE =====\n");
        printf("1. Insert Symbol\n");
        printf("2. Search Symbol\n");
        printf("3. Display Symbol Table\n");
        printf("4. Delete Symbol\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                insertSymbol();
                break;

            case 2:
                search();
                break;

            case 3:
                displayTable();
                break;

            case 4:
                deleteSymbol();
                break;

```

```
        case 5:
            return 0;

        default:
            printf("Invalid choice.\n");
    }
}

return 0;
}
```